CSC 357: Programming Assignment – 1
Winter 2026

Overview

This assignment serves as an introduction to programming in C beyond the smaller exercises in the Lab assignments. Even so, as the first assignment, the programs that you will write are smaller than you may be used to and smaller than what is to come. Do not, however, let the size of the programs lead to complacency. Take the opportunity to emphasize a focused understanding, to explore the use of new tools to aid in your programming, and to reenforce concepts from prior coursework and projects. Challenge yourself to develop a quality solution that goes beyond a solution that just works.

Course Learning Objectives

This assignment addresses the following course learning objectives.

Direct

- Read and write complex programs in the C programming language.
- Transition to upper-division coursework where programming is a tool used in the exploration of course-specific content.
- Transition from following stepwise, prescribed decompositions to independent problem solving.

Indirect

- Use standard UNIX user-level commands and the UNIX software development environment.

Setup

Clone the repository.

- Accept the GitHub classroom invitation: https://classroom.github.com/a/NtxiN7Pu
- Clone the repository.

Tips

- Use a debugger to inspect your data or print the data to verify that your program has stored what you intended it to.
- Program incrementally. Verify that each component works before integrating them.

The Tasks

There are two separate tasks listed below. Each task asks that you write a program to solve a "small" problem (each emulates a program commonly found in Unix installations) and while they are different programs, there is some (a little) common functionality between them.

Task 1 - word count

The `wc` program is a common utility available in many Unix installations. This program is rather simple in concept: it counts the number of characters, words, and lines in its input. Of course, to properly manage this we must adopt some definition of a "word" and of a "line" (the character count includes all characters in the input).

Definitions:

- word: whitespace delimited sequence of characters.
- line: sequence of characters terminated by a newline character.

Write a program (with `main` in a file named `word_count.c`) that takes a single (optional) command-line argument specifying a file name. The program must read the contents of the file (read from `stdin` if no file is specified) and print (only) the number of lines, number of words, and number of characters read (the format of the output is up to you). Use a single helper function to emulate the `wc` functionality.

Note: your program cannot dynamically allocate memory (e.g., by using one of the `*alloc` functions). It should read a single character at a time. Also, keep in mind that `wc` treats consecutive whitespace characters as a single delimiter when counting words.

You can verify the correctness of your program output by comparing your output with the output from the 'wc' command in Unix. For example, running the '**wc**' command with a sample demo.txt file produces the following output:

```
% wc demo.txt

    2     5    29 demo.txt
```

If the output of your own program confirms that demo.txt contains 2 lines, 5 words and 29 characters, your program is behaving correctly. You should also similarly verify your program output by comparing it with the '**wc**' output when '**wc**' is reading text from stdin.

Task 2 - unique lines

The `uniq` program is a common utility available in many Unix installations. This program echoes (to `stdout`) its input but will print only a single instance of consecutive matching lines. This is to say that if the input contained the following sequence of lines, then only a single instance of "This is a line." from the first group of matching lines will be printed, followed by "This is a different line.", and then "This is a line." (though it matches the first three, it was not part of the consecutive sequence).

This is a line.

This is a line.

This is a line.

This is a different line.

This is a line.

Write a program (with `main` in a file named `unique.c`) that takes a single (optional) command-line argument specifying a file name. The program must read the contents of the file (read from `stdin` if no file is specified) and print the contents of the input stream to `stdout` such that only a single instance of consecutive matching lines is printed.

Use a single helper function to emulate the `uniq` functionality.

Note: your program cannot explicitly dynamically allocate memory (e.g., by using one of the `*alloc` functions). It should use the `getline` function. A valgrind analysis of your program must not report any issues.

You can verify the correctness of your program output by comparing your output with the output from the 'uniq' command in Unix.

Deliverables

- Source Code - Push all relevant source code to your GitHub classroom repository.
- Canvas – After pushing to GitHub, update your GitHub username on Canvas under Project Assignment-1.

Please note that failing to provide your GitHub username on Canvas by the submission deadline will result in an incomplete submission which will not be graded. Submissions after the deadline will be counted as late submission (up to 7 days from the original deadline).

GRADING RUBRIC	
word count: functionality	35 points
word count: program style [The source code follows general best practices in terms of line length, function decomposition (using member functions and non-duplication of code), naming principles, documentation, etc.]	5 points
uniq: functionality	50 points
Uniq: program style [The source code follows general best practices in terms of line length, function decomposition (using member functions and non-duplication of code), naming principles, documentation, etc.]	5 points
uniq: clean valgrind report	5 points
Total	100 points